University of Haifa

Department of Information Systems

Software Engineering and Software Quality Assurance

Software Requirements Specification (SRS)

Educational Ludo Game

Based on IEEE Std 830-1998

Assignment 2 – Software Requirements Specification

Group Name: Sharks

Team Members:

Shafeek Saif	Rakan Ziyan	Mohamed Zereny
211614821	**324849249**	**327892972**

Table of Contents

1. Introduction.....1
1.1 Purpose.....1
1.2 Scope.....1
1.3 Definitions, Acronyms, and Abbreviations.....1
1.4 References.....3
1.5 Document Overview and Organization.....3
2. Overall Description.....4
2.1 Product Perspective.....4
2.2 Product Functions.....4
2.3 User Characteristics.....4
2.4 Constraints.....5
2.5 Assumptions and Dependencies.....5
2.6 Apportioning of Requirements.....6
3. Specific Requirements.....6
3.1 External Interface Requirements.....6
3.2 Functional Requirements.....7
3.3 Performance Requirements.....14
3.4 Design Constraints.....15
3.5 Software System Attributes.....15
3.6 Other Requirements – Logical Database Requirements.....17
4. Team Contribution.....18

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the functional and non-functional requirements of the Educational Ludo Game system. The document describes the expected behavior, features, interfaces, and constraints of the system, based on the project requirements provided by the course staff (R-2, R-4).

This SRS serves as a reference for the development team, the course staff acting as the customer, and the software testing team. It establishes a clear foundation for system design, implementation, and verification, following the guidelines of IEEE Std 830-1998 (R-1).

1.2 Scope

The Educational Ludo Game is a computer-based board game designed for students studying Software Engineering and Software Quality Assurance. The system combines traditional Ludo gameplay with educational multiple-choice questions related to the course material, providing an interactive and engaging learning experience.

The system supports two to four players on one shared computer and three difficulty levels: Easy, Medium, and Hard, each with a different board configuration. Players roll dice, move their pieces, and interact with question, surprise, and protection squares. Completed games are recorded in a game history, and an administrator can securely manage the educational questions. Online play, computer-controlled players, saving unfinished games, and mobile versions are outside the scope of this version.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Active Player	The player whose turn it is.
Administrator	Authorized user managing the question bank.
Arrow Square	Square that turns pieces of its color inward.
Boost / UltraBoost	Special dice affecting movement according to level.
Capturing	Returning a lone opponent piece to its castle.

Castle	Starting area holding a player's four pieces.
CSV	Comma-Separated Values file format.
Design Pattern	Proven, reusable solution to a design problem.
Double Block	Two or more same-player pieces blocking a square.
Feature	A system service used through a sequence of actions.
Finishing Square	Last square of the home path, where a piece finishes.
FR / NFR	Functional / Non-Functional Requirement.
Game History	Stored results of completed games.
Game Piece	Moveable token controlled by a player.
GUI	Graphical User Interface.
Hash / Salt	One-way password code / random value added first.
Home Path	The final, player-specific path to the finishing square.
IEEE 830	Standard providing guidelines for SRS documents.
JAR	Single runnable file containing the Java program.
Javadoc	Standard format for Java code documentation.
Legal Move	A move allowed by all rules in section 3.2.1.
MVC	Model-View-Controller software architecture.
PBKDF2 / AES	Standard Java methods for password hashing / encryption.
Player	User who plays Ludo and answers questions.
Pri	Priority (1 = Low, 2 = Medium, 3 = High).
Protected Square	A square where pieces cannot be captured.
Protection Square	Star-marked protected square placed at random.
Question Bank	Collection of questions organized by difficulty level.
Question Square	A square that triggers a multiple-choice question.
R-n	Reference number n in section 1.4; (R-4, Qn) is question n.
Regular Die	Standard die with values 1–6, used on all levels.
Rule of 6	Extra roll after a 6; a third 6 in a row is canceled.
Shall	Marks a mandatory requirement.
Shared Track	The common path followed by players' pieces.
Should	Marks a desirable, optional requirement.
Special Die	The Boost or UltraBoost die.
SRS	Document specifying software requirements.

Starting Square	Protected square where pieces enter from the castle.
Stimulus/Response	A user action and the system's reaction to it.
Surprise Square	A square that activates a random gameplay effect.
Swing	Standard Java library for graphical interfaces.
Unit Test	Automatic test of one small part of the code.
User Class	A group of users with the same role and permissions.

1.4 References

The following sources were used to prepare this SRS:

R-1 IEEE. IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications, 1998.

R-2 University of Haifa. Ludo Project – Group Project Description, 2026.

R-3 University of Haifa. Assignment 2 – SRS, October 19, 2026.

R-4 Course Staff. ANSWERS.xlsx – Staff Clarifications, 2026.

R-5 Course Staff. History-DataBase Template – Game.csv, 2026.

R-6 Sharks Team. Assignment 1 – Initial Entities and Requirements, 2026.

R-7 University of Haifa. Practice Lesson 2 – Scrum Methodology & SRS, 2026.

R-8 Anthropic. Claude (AI assistant), 2026. Used to help draft and edit the text; all requirements were reviewed and approved by the team.

1.5 Document Overview and Organization

This SRS follows IEEE Std 830-1998 (R-1) and consists of three main sections: Introduction, Overall Description, and Specific Requirements, followed by the team contribution.

Selected Structure and Justification. Section 3 follows the combined User Class and Feature structure (R-1, Annex A.8). This approach was selected because the system has two distinct user classes: Players, who interact with gameplay features, and Administrators, who manage the question bank only after authentication. Grouping by user class makes this access boundary explicit, and grouping by feature within each class matches how the system is used, as sequences of user actions and system responses.

Other structures fit less well: mode (A.1, A.2) would repeat shared rules for every level; user class alone (A.3) would leave fifty player requirements unstructured; feature alone (A.5) would hide the access boundary; object (A.4) would split multi-

object rules such as capturing; stimulus (A.6) would mix unrelated rules; and functional hierarchy (A.7) suits data-processing systems.

Identifiers and Priorities. Requirements are identified as FR-nn and NFR-nn. Each FR priority follows the IEEE 830 degree of necessity (R-1, 4.3.5.2): 3 = High (essential), 2 = Medium (improves the system, which is acceptable without it), 1 = Low (optional). "Shall" marks mandatory and "should" desirable requirements.

Non-Applicable Sections. These are kept with the reason: items of 2.1, sections 3.1.2, 3.1.4, and 3.5.2.

2. Overall Description

2.1 Product Perspective

The Educational Ludo Game is a new, standalone Java application developed for the course. It is not part of a larger system and runs on one computer shared by all players (R-4, Q20).

1. **System Interfaces:** Not relevant; no data is exchanged with other systems.
2. **User Interfaces:** A GUI with the screens listed in section 3.1.1.
3. **Hardware Interfaces:** Standard input and display devices (section 3.1.2).
4. **Software Interfaces:** Java runtime and file system (section 3.1.3).
5. **Communications Interfaces:** Not relevant; the game runs locally (3.1.4).
6. **Memory Constraints:** 4 GB of memory; all data files below 5 MB.
7. **Operations:** Gameplay and question management; no backup is required.
8. **Site Adaptation:** Not relevant; the same JAR file runs on every computer.

2.2 Product Functions

The Educational Ludo Game provides seven main functions: game setup for 2–4 players with names, unique colors, and difficulty; board configuration by difficulty level; turn and dice management; piece movement, capturing, blocking, and finishing; educational questions and surprise events; game completion, rankings, and CSV history; and secure administrator management of questions. Detailed requirements are specified in Section 3.

2.3 User Characteristics

The Educational Ludo Game supports two main user types, each with different responsibilities and access permissions.

1. **Player:** Students of the course who configure and play games, answer questions, and view game history. They need no technical knowledge and may not know Ludo, so the system enforces all rules and allows only legal moves.
2. **Administrator:** An authorized user, such as a course staff member, who manages the question bank through forms and needs no programming knowledge. Access is restricted to protect the questions. An administrator may also play as a regular player (R-4, Q23).

2.4 Constraints

The development of the Educational Ludo Game is subject to the following constraints defined in the official project specification:

1. **Programming Language:** The system must be implemented using Java, as required by the course staff (R-2).
2. **Software Architecture:** The application must follow MVC and use appropriate Design Patterns (R-2).
3. **Data Storage:** Game history must use the predefined CSV schema, preserving field names and order (R-5; R-4, Q48).
4. **Security and Access Control:** Question management must be accessible only to administrators; the mechanism is justified in section 3.5.3.
5. **Gameplay Rules:** The system must comply with the official Ludo project specification and the staff clarifications (R-2, R-4).
6. **Delivery:** Each version must be a runnable JAR file tested on more than one computer (R-2).
7. **Regulatory Policies:** None apply; the only personal data are player names, stored locally.

2.5 Assumptions and Dependencies

Assumptions:

1. **Local Gameplay:** The game runs on a single computer, with all players participating through the same interface (R-4, Q20).
2. **Player Turn Order:** Players take turns according to the order in which they were entered during game setup (R-4, Q79).
3. **Interrupted Games:** Only completed games are recorded in the game history (R-4, Q86).

4. **Open Rule Questions:** A 6 reaching the "extra roll" surprise grants both rolls, surprise roll first; backward movement retraces the route across arrow squares; and the unshown Hard board corners mirror the shown one.

Dependencies:

1. **Java Runtime:** Java 17 or later and a writable program folder.
2. **Question Bank:** At least 30 valid questions covering the three levels.
3. **Display:** A screen of at least 1280×720 pixels.
4. **Official Specifications:** Staff requirements and clarifications (R-2, R-4) take precedence; assumptions will be reviewed if they change.

2.6 Apportioning of Requirements

All mandatory project requirements (priorities 3 and 2) are allocated to the current version of the Educational Ludo Game. Optional requirements of priority 1 (FR-50, FR-55, and FR-62) may be delayed to a later iteration if needed, and the out-of-scope features of section 1.2 are reserved for future releases.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system shall provide a graphical user interface (GUI) with the following screens:

- **Main Menu:** New game, game history, question management, and exit.
- **Game Setup:** Number of players, difficulty, names, and colors.
- **Game Board:** Board, pieces, current player, time, dice, and Roll.
- **Question and Surprise Screens:** Questions, answers, and surprises.
- **Game Results:** Final rankings, difficulty, and game duration.
- **Game History:** Records of completed games.
- **Administrator Interface:** Login and question management screens.

3.1.2 Hardware Interfaces

Not relevant. The system requires no dedicated hardware interfaces and uses standard computer input and display devices through the operating system.

3.1.3 Software Interfaces

The application requires the Java runtime environment, version 17 or later, and uses the file system for the CSV game history (section 3.6), the question bank, and the administrator account. No database or external software is required.

3.1.4 Communication Interfaces

Not relevant. No network communication interfaces are required, since the system operates locally on one shared computer (R-4, Q20).

3.2 Functional Requirements

The following requirements are organized by user class and system feature, according to IEEE 830 Annex A.8 (R-1). Each functional requirement has a unique identifier, a priority level (Pri: 1 = Low, 2 = Medium, 3 = High), and a brief justification.

3.2.1 User Class: Player

3.2.1.1 Feature: Main Menu and Navigation

Purpose: Provide access to the main system functions.

Stimulus/Response: Application launch → Main menu displayed; Option selected → Corresponding screen or authentication process.

ID	Functional Requirement	Pri	Justification
FR-01	The system shall display a main menu with New Game, Game History, Question Management, and Exit options.	3	Essential: all functions start here.
FR-02	Selecting an option shall open the corresponding screen; Question Management shall first require authentication (FR-51).	3	Essential system navigation.
FR-03	Every screen shall offer a Back option; leaving a running game shall require confirmation, and the game is not saved.	2	Prevents accidental loss; not essential.

3.2.1.2 Feature: Game Setup

Purpose: Configure a new game before gameplay begins.

Stimulus/Response: Player enters game settings → System validates inputs → Game starts if valid.

ID	Functional Requirement	Pri	Justification
FR-04	The system shall allow selection of 2, 3, or 4 players.	3	Required player range.
FR-05	The system shall allow selection of Easy, Medium, or Hard difficulty.	3	Essential: sets the board and dice.

FR-06	The system shall require a unique name or nickname of 1–12 characters for each player.	3	Essential for rankings.
FR-07	Each player shall select a unique color from Red, Green, Yellow, and Blue.	3	Essential to tell pieces apart.
FR-08	The system shall prevent game initialization when required settings are missing or invalid, and indicate the invalid field.	3	Essential: invalid setups fail.
FR-09	Player names, colors, and difficulty shall not change after the game starts (R-4, Q95, Q102).	2	Keeps games consistent.

3.2.1.3 Feature: Game Board Configuration

Purpose: Generate and display the appropriate game board based on the selected difficulty level.

Stimulus/Response: Game settings confirmed → Board generated with the required tracks, squares, and player pieces.

Table 1 – Board Configuration by Difficulty Level (R-2)

Feature	Easy	Medium	Hard
Shared Track	52 squares	52 + 64 squares	52 + 64 + 72 squares
Home Path	6 per color	6 per color	6 per color
Protection Squares	4	8	12
Arrow Squares	4	8	12
Starting Squares	4	4	4
Question Squares	0	6 (2 per level)	9 (3 per level)
Surprise Squares	0	2	3
Dice	Regular	Regular + Boost	Regular + UltraBoost

ID	Functional Requirement	Pri	Justification
FR-10	The system shall generate the board of the selected level as specified in Table 1 and the official diagrams (R-2).	3	Essential board structure.
FR-11	The system shall include the required number of special squares for each difficulty level, as defined in Table 1 (R-4, Q92).	3	Required by the specification.

FR-12	Protection, question, and surprise squares shall be placed randomly on shared-track squares at game start, never overlapping (R-4, Q25).	3	Required: a new board in every game.
FR-13	The system shall initially place four game pieces in each player's castle.	3	Essential initial game state.
FR-14	The system shall display the board, tracks, pieces, and square types using distinct symbols.	2	Helps players understand the board.

3.2.1.4 Feature: Turn and Dice Management

Purpose: Manage player turns, dice rolls, movement calculations, and additional turns according to the game rules.

Stimulus/Response: Active player rolls dice → System calculates movement steps → Player performs a legal move or the turn advances according to the rules.

ID	Functional Requirement	Pri	Justification
FR-15	The system shall enforce the setup turn order and allow only the active player to roll dice.	3	Essential turn rule.
FR-16	Dice results shall be uniformly random: Regular (1–6) for all levels, Boost (1–3) for Medium, and UltraBoost (1–6) for Hard.	3	Essential for fair play.
FR-17	Movement shall equal the Regular die in Easy, or Regular × (special die − 1) in Medium and Hard; a result of 0 moves no piece.	3	Essential movement formula.
FR-18	A piece on its home path shall move by the Regular die result only.	3	Essential: prevents stuck pieces.
FR-19	A Regular 6 shall allow placing a piece on its starting square, ignoring the special die, or moving an active piece, unless an opponent Double Block occupies that square (R-4, Q40).	3	Essential castle exit rule.
FR-20	Each roll shall move exactly one piece; steps shall not be split between pieces, and a legal move may not be skipped (R-4, Q12, Q47).	3	Enforces a core movement rule.

FR-21	A Regular 6 shall grant an extra roll even if movement is impossible or zero, except under FR-22; capturing or finishing grants none (R-4, Q57).	3	Essential extra-roll rule.
FR-22	Three consecutive Regular 6s shall cancel the third roll and end the player's turn (R-4, Q45).	3	Required limit on extra rolls.
FR-23	If no legal move is available and no additional roll is granted, the system shall display a message and pass the turn (R-4, Q69).	3	Essential: the game must continue.
FR-24	After a roll, the system shall highlight the pieces that can make a legal move.	2	Improves usability; not essential.

3.2.1.5 Feature: Piece Movement, Capturing and Blocking

Purpose: Enforce legal piece movement, capturing, blocking, and finishing rules.

Stimulus/Response: Player selects a piece → System validates the move → Piece moves according to the game rules.

ID	Functional Requirement	Pri	Justification
FR-25	Pieces shall move clockwise, follow only arrows of their color, and treat other arrows as regular squares (R-4, Q39).	3	Essential movement paths.
FR-26	Landing on an unprotected square with one opponent piece shall return it to its castle, after any type of movement (R-4, Q41).	3	Essential capturing rule.
FR-27	Protection, starting, surprise, and finishing squares shall prevent capturing and allow pieces from different players to coexist (R-4, Q54, Q80).	3	Essential protection rule.
FR-28	Two or more same-player pieces on one square shall form a Double Block, which the owner's pieces may cross or join (R-4, Q19).	3	Essential blocking rule.
FR-29	An opponent's Double Block shall prevent landing or passing in either direction; if no other piece can move fully, movement stops before it (R-4, Q87).	3	Required block restriction.
FR-30	A Double Block shall remain while two pieces share its square; its owner must break it if no other legal move exists (R-4, Q85).	2	Clarifies a case implied by FR-20.

FR-31	Finishing shall require exact steps; if a move overshoots, another legal piece must move, otherwise the turn passes (R-4, Q77).	3	Essential finishing rule.
FR-32	Backward movement caused by questions or surprises shall stop at the player's starting square without crossing it (R-4, Q50).	3	Required backward limit.
FR-33	A home path shall contain no special squares and only pieces of its own color (R-4, Q33).	3	Essential for a fair finish.

3.2.1.6 Feature: Educational Questions and Surprise Events

Purpose: Manage educational questions and surprise events during gameplay.

Stimulus/Response: A piece lands on a question or surprise square → The system activates the corresponding effect.

ID	Functional Requirement	Pri	Justification
FR-34	Landing on a question square shall display to all players a random question of matching difficulty with four options; only the active player may answer (R-4, Q55).	3	Essential educational goal.
FR-35	Correct answers shall move the piece forward 1 (Easy), 2 (Medium), or 3 (Hard) steps; incorrect answers move it back equally.	3	Required answer effects.
FR-36	After an answer, the system shall show whether it was correct and display the correct answer (R-4, Q63).	2	Supports learning; not essential.
FR-37	A question shall not repeat in the same game until all questions of its level have been asked (R-4, Q13).	2	Adds variety; not essential.
FR-38	Capturing on an unprotected question square shall occur before the question and shall not be reversed by the answer's effect (R-4, Q44).	3	Required event order.
FR-39	Landing on a surprise square shall trigger exactly one randomly selected outcome according to Table 2.	3	Essential surprise rule.

FR-40	For "move an opponent", the active player shall move an opponent piece on a shared track back by half a Regular die, rounded up, or cancel it if none exists (R-4, Q93).	3	Required surprise outcome.
FR-41	Question and surprise movements shall obey movement and blocking rules, except for the direct-finish surprise (R-4, Q6, Q52).	3	Required rule consistency.
FR-42	Question or surprise movement shall activate any new special square reached; zero-step rolls shall not reactivate a square (R-4, Q81).	3	Handles required chained effects.

Table 2 – Surprise Event Probabilities (R-2)

Probability	Surprise Outcome
10%	Jump to the finishing square, ignoring exact steps and blocks.
10%	Move an opponent piece back half a Regular die, rounded up.
10%	Return the piece to its castle.
30%	Move the piece backward three steps.
30%	Skip the next turn and cancel any pending 6 roll (R-4, Q56).
10%	Extra Regular + UltraBoost roll, outside the 6 rule (R-4, Q27).

3.2.1.7 Feature: Game Completion and History

Purpose: Determine game completion, player rankings, and game history records.

Stimulus/Response: Players finish their pieces → System updates rankings → Game ends → Results are displayed and saved.

ID	Functional Requirement	Pri	Justification
FR-43	A player shall finish when all four pieces reach the finishing square and shall take no further turns (R-4, Q103).	3	Essential for completion.
FR-44	The game shall end when all but one player have finished; the remaining player receives the last rank (R-4, Q49).	3	Essential game-ending rule.
FR-45	Rankings shall follow the order in which players finish their four pieces.	3	Essential for final results.

FR-46	The system shall measure and display elapsed game time in HH:MM:SS format.	3	Required for history.
FR-47	At game completion, the system shall display rankings, difficulty level, and game duration.	2	Provides result visibility.
FR-48	Completed games shall be saved in CSV format as specified in section 3.6; unfinished games shall not be saved (R-4, Q86).	3	Required CSV format.
FR-49	The system shall allow players to view previously completed games, newest first, through the Game History screen.	3	Required history access.
FR-50	The Game History screen should allow filtering games by difficulty level.	1	Optional convenience.

3.2.2 User Class: Administrator

3.2.2.1 Feature: Administrator Authentication

Purpose: Restrict question management to authorized administrators.

Stimulus/Response: Question Management selected → Login form displayed; Credentials entered → Access granted or error displayed.

ID	Functional Requirement	Pri	Justification
FR-51	Question Management shall require administrator authentication with a username and password before access.	3	Prevents unauthorized access.
FR-52	The system shall provide one predefined administrator account, whose credentials are delivered to the course staff (R-4, Q91).	3	Required for secure access.
FR-53	Failed authentication shall display a general error message, without revealing which credential was wrong, and grant no access.	3	Essential for security.
FR-54	Leaving the administrator interface shall end the administrator session.	2	Improves security; not essential.
FR-55	The administrator should be able to change the password.	1	Optional security improvement.

3.2.2.2 Feature: Question Management

Purpose: Allow the administrator to maintain the educational question bank.

Stimulus/Response: Administrator selects an action → System displays the question list or form → Changes are validated and saved.

ID	Functional Requirement	Pri	Justification
FR-56	The question bank shall contain at least 30 course-related questions across Easy, Medium, and Hard levels (R-2).	3	Required by the project.
FR-57	The system shall display all questions with their text, difficulty level, and correct answer.	3	Required to manage questions.
FR-58	The administrator shall be able to add a question with text, four different answer options, one correct answer, and a difficulty level; incomplete questions shall be rejected.	3	Required question management.
FR-59	The administrator shall be able to edit any field of an existing question, with the same validation.	3	Required question management.
FR-60	The administrator shall be able to delete a question after confirmation, unless fewer than 30 questions would remain.	3	Required; keeps enough questions.
FR-61	Changes to the question bank shall be saved immediately and apply from the next game (R-4, Q53).	3	Essential: no lost changes.
FR-62	The question list should support filtering questions by difficulty level.	1	Optional convenience.

3.3 Performance Requirements

The following limits apply on a reference computer with a dual-core processor, 4 GB of memory, a 1280×720 screen, and Java 17. They were set by the team: actions repeated throughout play (rolling, moving, opening a question or surprise, saving) must respond within 1–2 seconds so the game flows without noticeable waiting, while one-time actions (starting the program, building the board) may take slightly longer.

ID	Non-Functional Requirement
NFR-01	The main menu shall appear within 5 seconds of starting the program.
NFR-02	The game board shall appear within 2 seconds of starting a game, including random square placement.
NFR-03	Dice results shall appear within 1 second of rolling, including any animation.
NFR-04	Each piece movement shall be animated square by square and complete within 2 seconds.
NFR-05	Question and surprise screens shall open within 1 second after the piece stops.
NFR-06	Saving a game shall take at most 1 second; Game History shall open within 2 seconds for up to 1,000 games.

3.4 Design Constraints

ID	Non-Functional Requirement
NFR-07	The system shall be implemented in Java 17 with the Swing library, which is included in standard Java, so no extra installation is needed (R-2).
NFR-08	The code shall follow MVC: Model classes hold data and game rules without GUI code, View classes contain no game rules, and Controller classes connect them (R-2).
NFR-09	Documented Design Patterns shall include Factory Method (boards), Observer (screens), State (turn phases), and Singleton (history file) (R-2).
NFR-10	Each version shall be delivered as a runnable JAR file and tested on at least two computers before submission (R-2).

3.5 Software System Attributes

3.5.1 Reliability

ID	Non-Functional Requirement
NFR-11	Invalid input or unexpected game situations shall not stop the program; invalid input shall be rejected with an explanatory message.
NFR-12	A missing history file shall be recreated with its header row; damaged records shall be skipped and reported.
NFR-13	In 10,000 simulated draws, each die value and surprise outcome shall be within 2 percentage points of its expected probability.

NFR-14	Every movement and special square rule shall be covered by unit tests, including edge cases, and all tests shall pass before each delivery.

3.5.2 Availability

Not relevant. The system is an offline desktop application without a server, so it is available whenever it is started.

3.5.3 Security

Selected mechanism: password authentication with salted PBKDF2 hashing, a lockout after failed attempts, and an encrypted question file. Since players and the administrator share one computer, both the interface and the stored data must be protected, using standard Java libraries only. The 30-second lockout after 3 failed attempts slows password guessing without hindering the administrator.

ID	Non-Functional Requirement
NFR-15	The question management interface shall open only after successful authentication; an administrator playing a game has no administrator rights during it.
NFR-16	The administrator password shall contain at least 8 characters, including a letter and a digit, and shall be stored only as a salted PBKDF2 hash.
NFR-17	After 3 consecutive failed login attempts, login shall be disabled for 30 seconds.
NFR-18	The question bank file shall be encrypted with AES, using a key stored inside the program rather than in the data folder.

3.5.4 Maintainability

ID	Non-Functional Requirement
NFR-19	Level settings (track sizes, special square counts, dice) and surprise probabilities shall be defined in one configuration class.
NFR-20	Model, View, and Controller classes shall be in separate packages, with Javadoc for public classes and methods.

3.5.5 Portability

ID	Non-Functional Requirement
NFR-21	The same JAR file shall run without changes on Windows 10 or 11 and macOS 13 or later, with Java 17 or later.

NFR-22	Data files shall be located relative to the program folder, without fixed disk paths.

3.5.6 Usability

ID	Non-Functional Requirement
NFR-23	The whole board shall be visible without scrolling on screens of 1280×720 pixels or more and shall scale with the window (R-4, Q21).
NFR-24	Text shall be at least 12 pt, and each player's color shall also be identified by the player's name.
NFR-25	At least 3 of 4 first-time users shall be able to set up and start a game within 60 seconds without help.

3.6 Other Requirements – Logical Database Requirements

The system stores three data files: the game history, the question bank, and the administrator account. History records use the staff-provided template (R-5).

Required CSV Fields (in order):

gameID, difficulty, timestamp(DD/MM/YYYY; HH:MM:SS), 1st_place_name, 2nd_place_name, 3rd_place_name, 4rd_place_name, game_duration(HH:MM:SS).

ID	Functional Requirement	Pri	Justification
FR-63	Each completed game shall be appended as one record with the required fields in order, leaving empty places blank (R-4, Q48).	3	Required template format.
FR-64	Each question shall be stored with a unique ID, its text, four answer options, the correct answer, and its difficulty level.	3	Required to display and edit questions.
FR-65	The administrator account shall store only the username, the salt, and the password hash.	3	Required for security.

4. Team Contribution

The following table describes the contribution of each team member to the parts of this document.

Team Member	Contribution to the SRS
Shafeek Saif	Section 1 – Introduction: purpose, scope, definitions, references, and document overview, including the justification of the selected IEEE 830 structure. Section 2 – Overall Description: product perspective, product functions, user characteristics, constraints, assumptions and dependencies, and apportioning of requirements.
Rakan Ziyan	Section 3.1 – External Interface Requirements. Section 3.2.1 – Player functional requirements (FR-01 to FR-50): main menu, game setup, board configuration (Table 1), turn and dice management, piece movement, capturing and blocking, questions and surprise events (Table 2), and game completion and history.
Mohamed Zereny	Section 3.2.2 – Administrator functional requirements (FR-51 to FR-62). Sections 3.3–3.6 – Performance requirements, design constraints, software system attributes (reliability, availability, security, maintainability, portability, usability), and logical database requirements (FR-63 to FR-65).
All members	Joint review of all requirements against the project specification and staff clarifications, priority ranking and justifications, consistency checks, and final editing of the document.